Лабораторная работа №2

Системы контроля версий Git

4 марта,2026

Содержание

Цель работы 4
Задание 4
Теоретическое введение 4
Выполнение лабораторной работы 5
Вывод 9
Контрольные вопросы 9
Список литературы 13

Список иллюстраций

1 Установка Git 5
2 Настройки Git 5
3 Создание SSH-ключей rsa и ed25519 6
4 Создание PGP ключа 6
5 Публичный SSH-ключ для добавления на GitHub 7
6 Экспорт PGP-ключа 7
7 Настройка подписи коммитов 7
8 Авторизация gh 8
9 Создание структуры каталогов 8
10 Создание структуры каталогов 8
11 Клонирование репозитория 8
12 НастройКа рабочей директории 9
13 Отправка изменений на GitHub 9

Список таблиц

Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

Задание

1. Создать базовую конфигурацию для работы с git.
2. Создать ключи SSH и PGP.
3. Настроить подписи коммитов.
4. Зарегистрироваться на GitHub.
5. Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

Системы контроля версий (VCS) позволяют отслеживать изменения в файлах, работать над проектами совместно и возвращаться к предыдущим версиям. Git — распределённая система контроля версий, которая хранит историю изменений локально и может синхронизироваться с удалёнными репозиториями.

Основные команды Git: - `git init` — инициализация репозитория - `git add` — добавление файлов в индекс - `git commit` — сохранение изменений - `git push` — отправка изменений на сервер - `git pull` — получение изменений с сервера

Выполнение лабораторной работы

На первом шаге нужно было выполнить установку Git,но как оказалось он у меня уже установлен.(рис. @fig-01)

Рис. 1: Установка Git

Произвожу базовую настройку git. (рис. @fig-02)

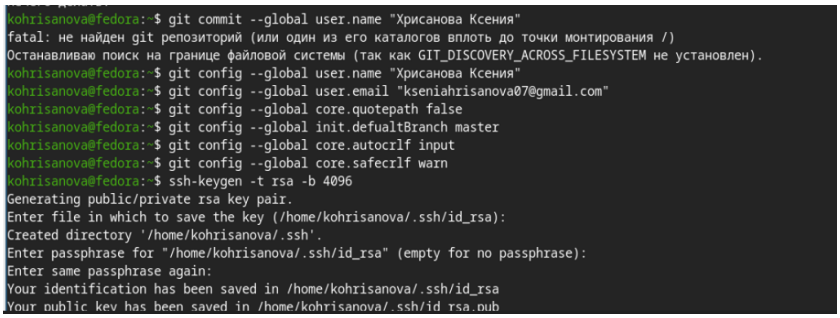
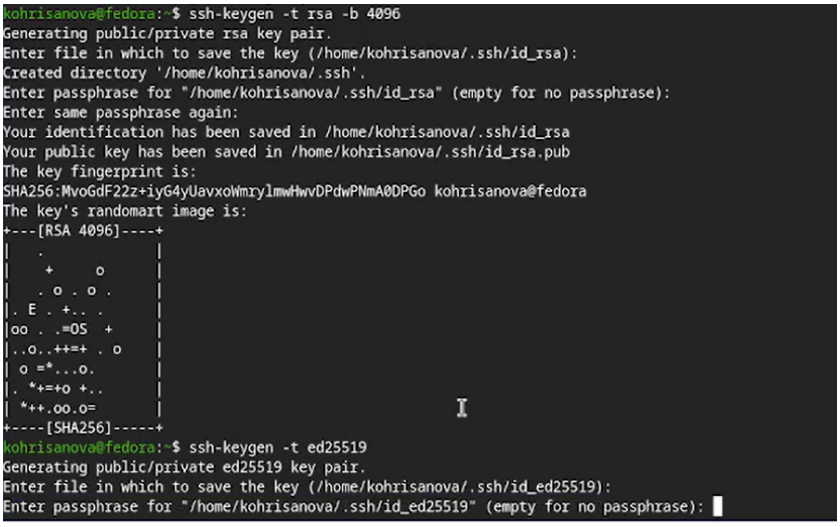


Рис. 2: Настройки Git

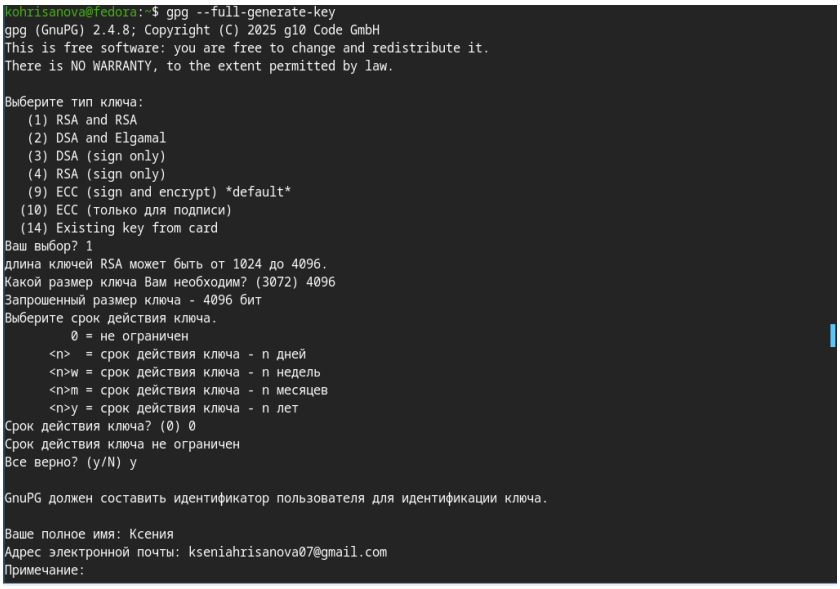
Создаю SSH ключи (рис. @fig-03)



```
kohrisanova@fedora:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kohrisanova/.ssh/id_rsa):
Created directory '/home/kohrisanova/.ssh'.
Enter passphrase for "/home/kohrisanova/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kohrisanova/.ssh/id_rsa
Your public key has been saved in /home/kohrisanova/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:MvoGdF22z+iyG4yUavxoWmrylmwHwvDPdwPNmA0DPGo kohrisanova@fedora
The key's randomart image is:
+---[RSA 4096]----+
|   .             |
|    +     o      |
|   . o . o .     |
|. E . +.. .      |
|oo . .=OS  +     |
|..o..++=+ . o    |
| o =*...o.       |
|. *+=+o +..      |
| *++.oo.o=       |
+----[SHA256]-----+
kohrisanova@fedora:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kohrisanova/.ssh/id_ed25519):
Enter passphrase for "/home/kohrisanova/.ssh/id_ed25519" (empty for no passphrase):
```

Рис. 3: Создание SSH-ключей rsa и ed25519

Создаю PGP ключ(рис. @fig-04)



```
kohrisanova@fedora:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Ксения
Адрес электронной почты: kseniahrisanova07@gmail.com
Примечание:
```

Рис. 4: Создание PGP ключа

После создания ключа я скопировала его и добавила в настройках GitHub (рис. @fig-05).

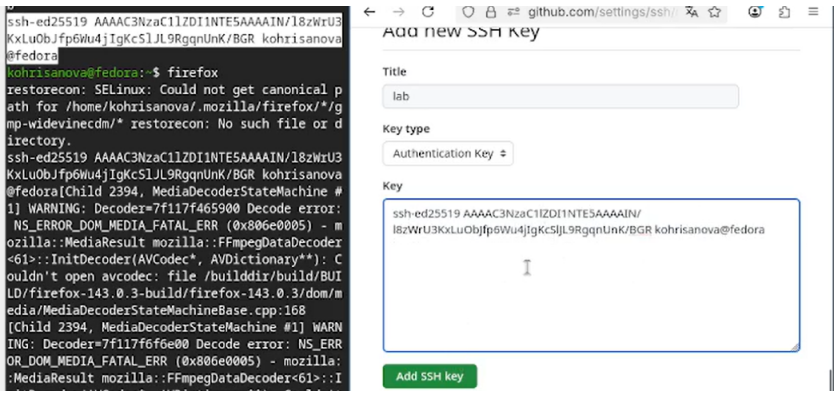


Рис. 5: Публичный SSH-ключ для добавления на GitHub

Экспортировала PGP-ключ и добавила на GitHub(рис. @fig-06)

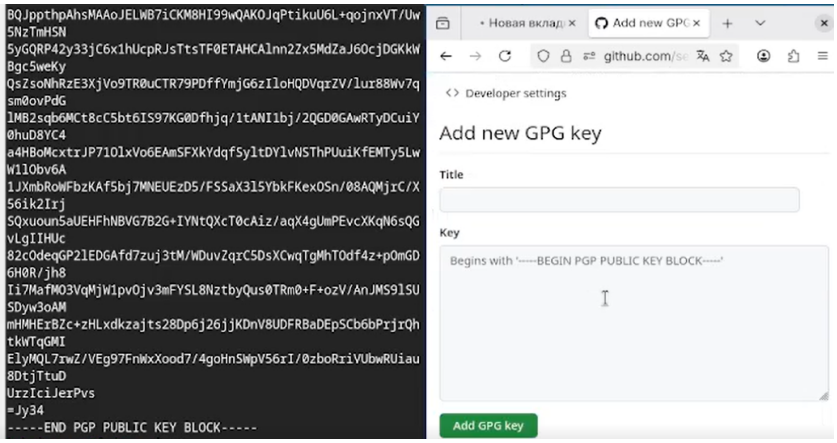


Рис. 6: Экспорт PGP-ключа

Настраиваю автоматические подписи для коммитов. (рис. @fig-07)

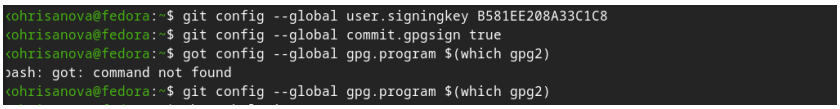


Рис. 7: Настройка подписи коммитов

Авторизуюсь на github для работы через терминал. (рис. @fig-08)

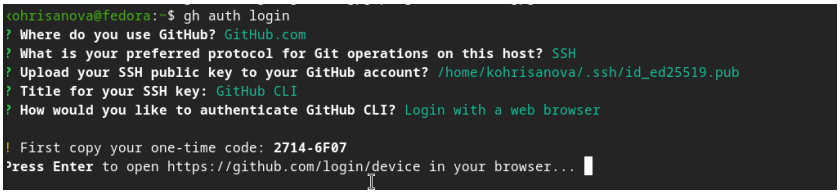


Рис. 8: Авторизация gh

Создаю структуру каталогов(рис. @fig-09)

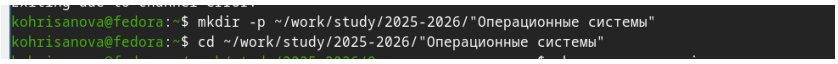


Рис. 9: Создание структуры каталогов

Создаю репозиторий на GitHub из шаблона(рис. @fig-10)

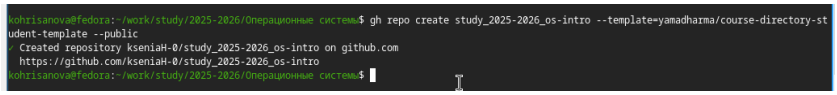


Рис. 10: Создание структуры каталогов

Клонирую репозиторий(рис. @fig-11)

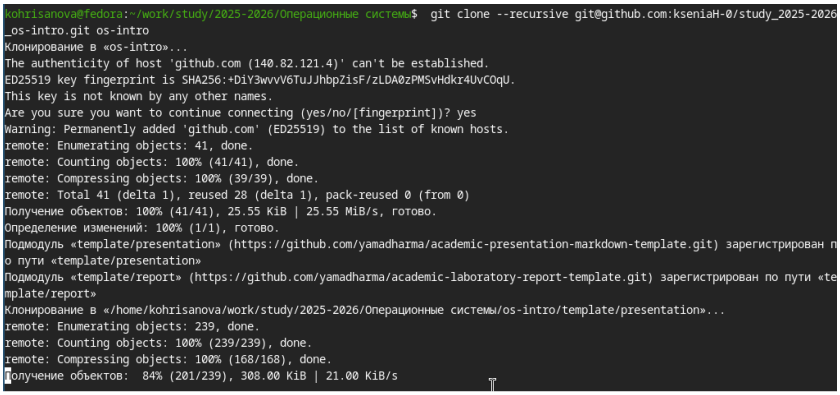


Рис. 11: Клонирование репозитория

Настраиваю рабочую директорию (рис. @fig-12)

```
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы$ cd os-intro
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ rm package.json
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-intro > COURSE
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ make
Usage:
  make <target>

Targets:
  list                            List of courses
  prepare                         Generate directories structure
  submodule                       Update submules

kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$
```

Рис. 12: НастройКа рабочей директории

Отправляю изменения на GitHub(рис. @fig-13)

```
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -am 'feat(main): make course structure'
[master f3d534a] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 4 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 1001 bytes | 1001.00 KiB/s, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:kseniaH-0/study_2025-2026_os-intro.git
   7da8792..f3d534a  master -> master
kohrisanova@fedora:~/work/study/2025-2026/Операционные системы/os-intro$
```

Рис. 13: Отправка изменений на GitHub

Вывод

В ходе выполнения лабораторной работы были изучены основные принципы работы систем контроля версий и получены практические навыки работы с Git. Выполнена базовая настройка Git, созданы SSH и PGP ключи, добавлены в настройки GitHub, настроена подпись коммитов. Освоена работа с GitHub CLI, создан репозиторий из шаблона и подготовлена структура каталогов. Цель работы достигнута.Работа выполнена в соответствии с методическими указаниямии[@lab2].

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий — это программные инструменты, которые помогают организовать совместную работу над проектами. Они хранят историю изменений файлов, позволяют отслеживать, кто и когда внёс правки, дают возможность возвращаться к предыдущим версиям, а также объединять изменения от разных участников. Основная задача VCS — обеспечить порядок и контроль при разработке, особенно когда над проектом работает несколько человек.

---

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

Хранилище — это место (локальное или удалённое), где хранятся все файлы проекта и история их изменений. Коммит — это зафиксированное состояние проекта в определённый момент времени, своего рода снимок того, как выглядят файлы после внесённых правок. История — это последовательность коммитов, которая позволяет проследить развитие проекта, увидеть, какие изменения вносились и кем. Рабочая копия — это текущая версия файлов проекта на компьютере разработчика, с которой он непосредственно работает; она может отличаться от последнего коммита, если изменения ещё не зафиксированы.

---

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные системы контроля версий работают по принципу «клиент-сервер»: все файлы и история хранятся в одном месте — на сервере, а разработчики подключаются к нему, чтобы получить актуальную версию или отправить свои изменения. Примерами таких систем являются CVS и Subversion. Распределённые системы устроены иначе: каждый участник имеет полную копию репозитория, включая всю историю. Это позволяет работать автономно, а изменения синхронизируются с другими копиями по мере необходимости. Центральный репозиторий здесь не обязателен, но часто используется как основное хранилище. К распределённым системам относятся Git и Mercurial.

---

4. Опишите действия с VCS при единоличной работе с хранилищем.

При работе в одиночку разработчик сам создаёт и ведёт репозиторий. Он может фиксировать изменения по мере выполнения задач, создавая коммиты с описанием того, что сделано. При необходимости всегда можно вернуться к более ранней версии, посмотреть историю или поработать с отдельной веткой, не затрагивая основную.

---

5. Опишите порядок работы с общим хранилищем VCS.

При работе в команде обычно используется центральный репозиторий, куда все загружают свои изменения. Каждый участник создаёт свою ветку для разработки новой функциональности, чтобы не мешать другим. Когда работа в ветке завершена, её изменения сливаются с основной веткой проекта. Перед этим полезно синхронизироваться с удалённым репозиторием, чтобы учесть правки коллег и избежать конфликтов.

---

6. Каковы основные задачи, решаемые инструментальным средством git?

Git решает три главные задачи. Во-первых, хранение истории — все изменения сохраняются, можно посмотреть, кто и когда что менял. Во-вторых, работа с версиями — позволяет сравнивать разные версии, объединять изменения, возвращаться к старым состояниям. Во-третьих, совместная работа — даёт возможность нескольким людям работать над одним проектом, синхронизируя свои изменения через общий репозиторий.

---

7. Назовите и дайте краткую характеристику командам git.

`git config` — задаёт параметры работы Git (имя, почта, редактор и т.д.).

`git status` — показывает, какие файлы изменены, добавлены или ждут коммита.

`git add` — помещает файлы в «индекс», подготавливая их к коммиту.

`git commit` — сохраняет текущее состояние проекта с комментарием.

`git branch` — выводит список локальных веток.

`git checkout` — переключает на другую ветку.

`git merge` — объединяет изменения из одной ветки с текущей.

`git push` — отправляет локальные коммиты на удалённый репозиторий.

`git pull` — загружает изменения с сервера и сливает их с локальной версией.

---

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

`git remote add origin https://...` — привязывает локальный репозиторий к удалённому.

`git remote remove origin` — отвязывает удалённый репозиторий.

`git remote rename origin upstream` — переименовывает удалённый репозиторий.

`git remote set-url origin https://...` — меняет адрес удалённого репозитория.

`git remote show origin` — показывает информацию о привязке и доступные ветки.

---

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветки нужны, чтобы работать над разными задачами независимо друг от друга. Например, можно создать отдельную ветку для новой функции и спокойно её разрабатывать, не рискуя сломать основную версию. После завершения работы ветку можно слить с главной. Ветка представляет собой просто указатель на один из коммитов, поэтому создавать их легко и быстро. Основная ветка обычно называется `main` или `master`, а остальные называют в соответствии с тем, что в них делается.

---

10. Как и зачем можно игнорировать некоторые файлы при commit?

В процессе разработки часто появляются файлы, которые не нужно отслеживать в Git: временные файлы, логи, настройки среды, скомпилированные программы и т.п. Чтобы они

случайно не попали в коммит, используется файл `.gitignore`. В нём можно перечислить названия или шаблоны файлов, которые Git будет игнорировать. Это помогает поддерживать репозиторий в чистоте и не засорять его служебными или сгенерированными данными.

Список литературы